

Programación de Aplicaciones Web (I)

Programación en Entornos de Red

GRADO EN INGENIERÍA BIOMÉDICA 2017-2018

Juan Gonzalez Gomez <juan.gonzalez.gomez@urjc.es>,
Agustín Santos <agustin.santos@urjc.es>

Verbos HTTP: GET

The diagram illustrates the components of an HTTP GET request. The request line is `GET/profile.jsp?user=abhi&course=java HTTP/1.1`. Annotations with arrows point to specific parts: 'Http request method' points to `GET`; 'Path to source on Web Server' points to `/profile.jsp`; 'Parameters' points to `?user=abhi&course=java`; and 'Protocol Version Browser support' points to `HTTP/1.1`. Below the request line, a large curly bracket labeled 'request header' groups the following lines: `Host: www.studytonight.com`, `User-Agent: Mozilla/5.0`, `Accept: text/xml,text/html,text/plain,image/jpeg`, `Accept-Language: en-us,en`, `Accept-Encoding: gzip`, `Keep-Alive: 300`, and `Connection: keep-alive`.

Http request method

Path to source on Web Server

Parameters

Protocol Version Browser support

GET/profile.jsp?user=abhi&course=java HTTP/1.1

request header

Host: www.studytonight.com

User-Agent: Mozilla/5.0

Accept: text/xml,text/html,text/plain,image/jpeg

Accept-Language: en-us,en

Accept-Encoding: gzip

Keep-Alive: 300

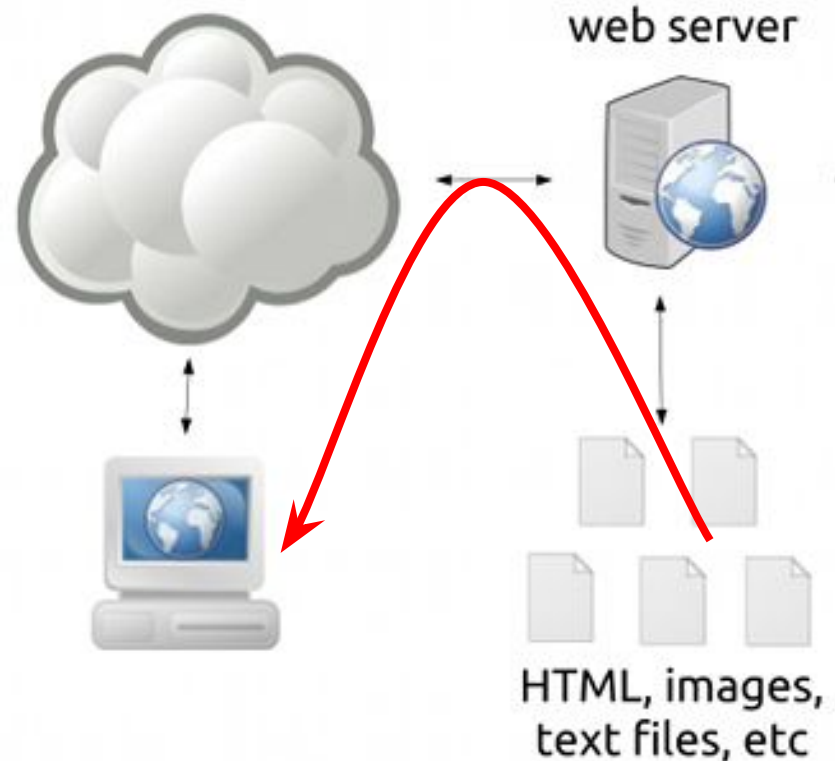
Connection: keep-alive

Lo usamos para obtener un fichero Html, texto, imagen, etc.

GET

Características:

- Cualquier tipo de fichero (html, texto, imagen, etc.)
- Se puede repetir sin efectos (es idempotente)
- Podemos dar parámetros o argumentos
- Una página Html puede invocar múltiples peticiones
- No mantiene la conexión
- Se puede “*cachear*”



GET



```
GET /hello.htm HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.tutorialspoint.com
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive
```



```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
ETag: "34aa387-d-1568eb00"
Vary: Authorization,Accept
Accept-Ranges: bytes
Content-Length: 88
Content-Type: text/html
Connection: Closed
```

```
<html>
<body>
<h1>Hello, World!</h1>
</body>
</html>
```

GET

Return to list view

Details tabs

The screenshot shows the Chrome DevTools Network tab. The top toolbar includes icons for Inspector, Console, Debugger, Style Editor, Timeline, and Network. The Network tab is active, and the 'Details' sub-tab is selected. A red box highlights the 'Details' sub-tab and the 'Headers' section. A red arrow points from the 'Return to list view' text to the list view icon in the Network tab. Another red arrow points from the 'Details tabs' text to the 'Details' sub-tab. The list of requests shows several GET requests to various files. The details of the first request are expanded, showing the Request URL, Request method, Status code, and Response headers.

✓	Method	File	
● 200	GET	The_Rite_of_Spring	en.wikiped
● 200	GET	load.php?debug=false&lang...	bits.wikim
● 200	GET	load.php?debug=false&lang...	bits.wikim
● 200	GET	load.php?debug=false&lang...	bits.wikim
● 200	GET	load.php?debug=false&lang...	bits.wikim
● 200	GET	15px-Cscr-featured.svg.png	upload.wik
● 200	GET	350px-NikolaiRoerichRite1.jpg	upload.wik
● 200	GET	170px-Igor_Stravinsky_as_dr...	upload.wik
● 200	GET	170px-Bakst_daighilev.jpg	upload.wik
● 200	GET	220px-BavliindmanClarenceS	upload.wik

Request URL: http://en.wikipedia.org/wiki/The_Rite_of_Spring
Request method: GET
Status code: ● 200 OK

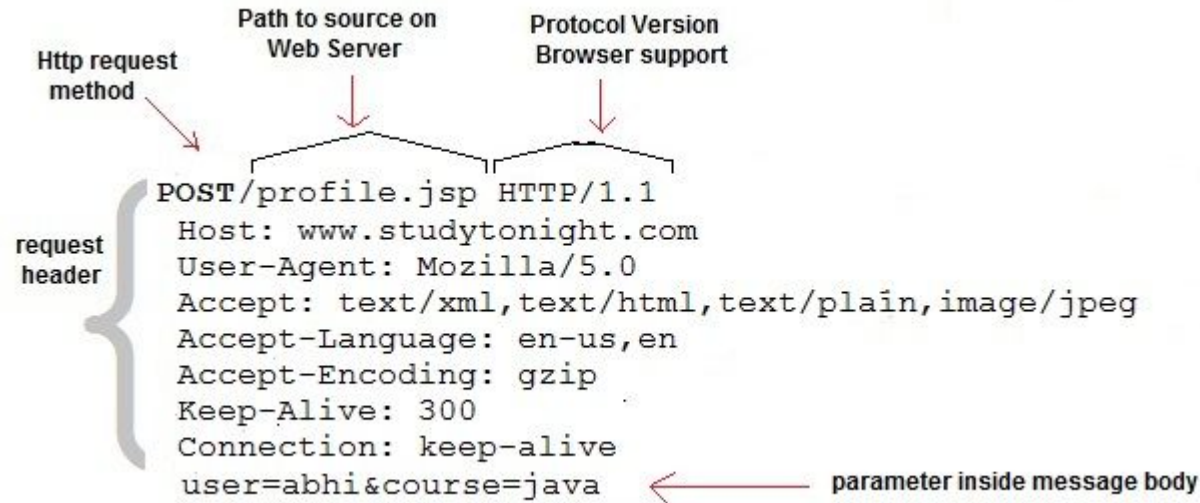
Filter headers

Response headers (0.640 KB)

- Accept-Ranges:** "bytes"
- Age:** "25857"
- Cache-Control:** "private, s-maxage=0, max-age=0, must-revalidate"
- Connection:** "keep-alive"
- Content-Encoding:** "gzip"
- Content-Language:** "en"

All HTML CSS JS XHR Fonts Images Media Flash Other 46 requests, 1,290.23 KB, 0.78 s Clear

Verbos HTTP: POST

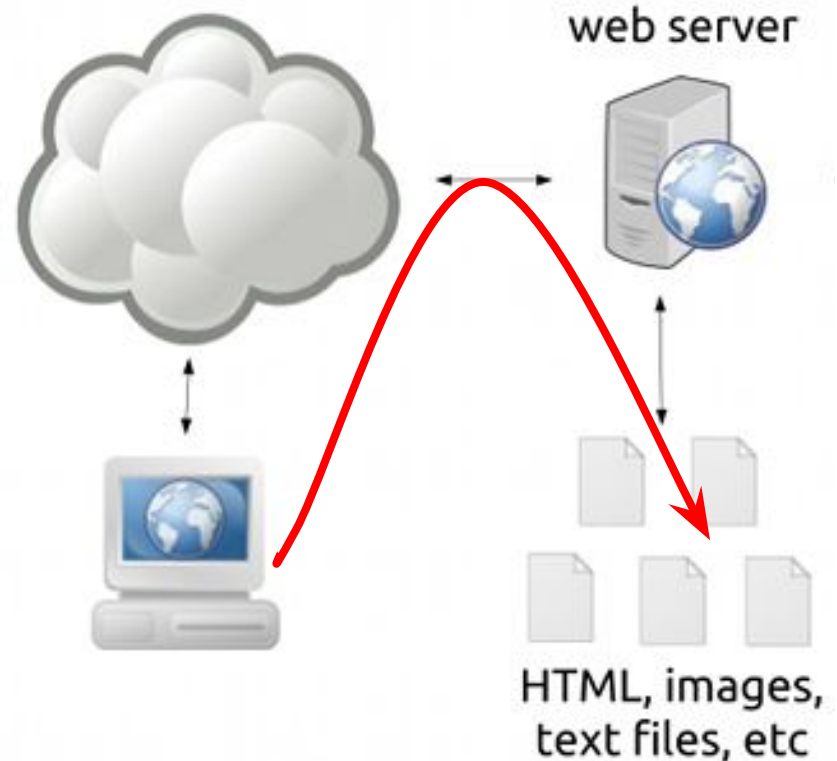


Lo usamos para mandar información al servidor

POST

Características:

- Lo usamos para mandar información
- No se puede “*cachear*”
- No es idempotente (puede tener efectos secundarios)
- En Html suele redirigir a otra página
- Los datos viajan en el cuerpo, no en los argumentos



POST



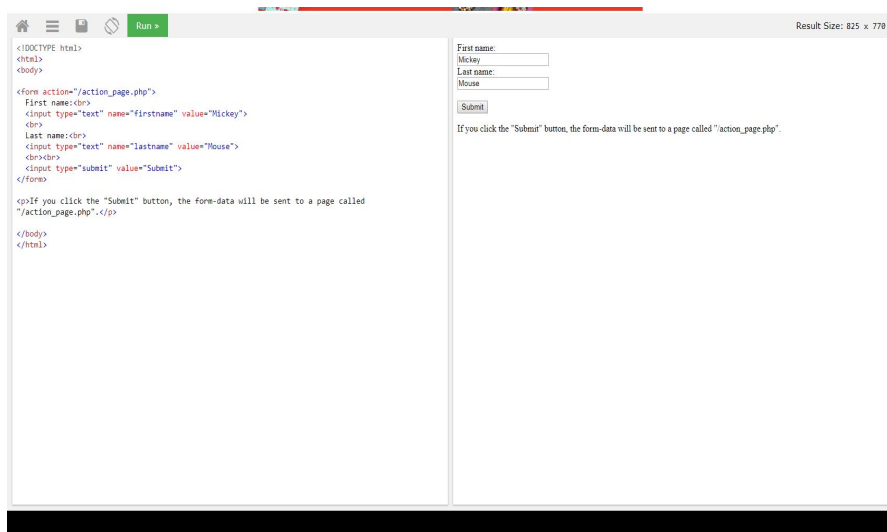
```
POST /cgi-bin/process.cgi HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.tutorialspoint.com
Content-Type: text/xml; charset=utf-8
Content-Length: 88
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive
<?xml version="1.0" encoding="utf-8"?>
<string xmlns="http://clearforest.com/">string</string>
```


POST



```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
ETag: "34aa387-d-1568eb00"
Vary: Authorization,Accept
Accept-Ranges: bytes
Content-Length: 88
Content-Type: text/html
Connection: Closed
<html>
<body>
<h1>Request Processed Successfully</h1>
</body>
</html>
```

Formularios HTML



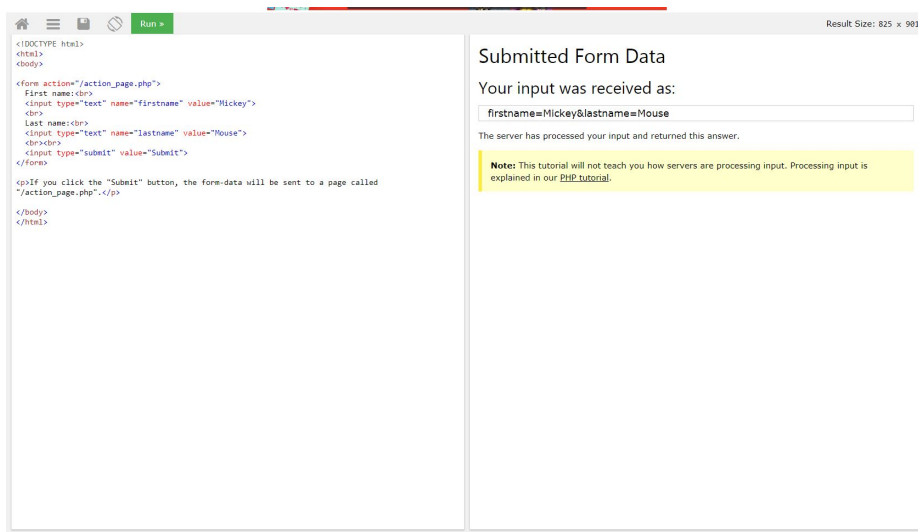
The screenshot shows a web browser window with a toolbar at the top. The main content area displays an HTML form. The form has two text input fields: "First name:" with the value "Mickey" and "Last name:" with the value "Mouse". Below these fields is a "Submit" button. A paragraph of text below the button reads: "If you click the 'Submit' button, the form-data will be sent to a page called 'action_page.php'." The browser's status bar at the bottom right indicates "Result Size: 825 x 778".

```
<!DOCTYPE html>
<html>
<body>

<form action="/action_page.php">
  First name:<br>
  <input type="text" name="firstname" value="Mickey">
  <br>
  Last name:<br>
  <input type="text" name="lastname" value="Mouse">
  <br><br>
  <input type="submit" value="Submit">
</form>

<p>If you click the "Submit" button, the form-data will be sent to a page called
"/action_page.php".</p>

</body>
</html>
```



The screenshot shows a web browser window displaying the result of a submitted form. The main content area has the heading "Submitted Form Data" and the text "Your input was received as:". Below this is a text box containing the string "firstname=Mickey&lastname=Mouse". A paragraph of text below the text box reads: "The server has processed your input and returned this answer." A yellow note box at the bottom right contains the text: "Note: This tutorial will not teach you how servers are processing input. Processing input is explained in our [PHP tutorial](#)." The browser's status bar at the bottom right indicates "Result Size: 825 x 901".

```
<!DOCTYPE html>
<html>
<body>

<form action="/action_page.php">
  First name:<br>
  <input type="text" name="firstname" value="Mickey">
  <br>
  Last name:<br>
  <input type="text" name="lastname" value="Mouse">
  <br><br>
  <input type="submit" value="Submit">
</form>

<p>If you click the "Submit" button, the form-data will be sent to a page called
"/action_page.php".</p>

</body>
</html>
```

https://www.w3schools.com/html/html_forms.asp

Código HTML Form

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<form action="/action_page.php" method="GET">
```

```
  First name:<br>
```

```
  <input type="text" name="firstname" value="Mickey">
```

```
  <br>
```

```
  Last name:<br>
```

```
  <input type="text" name="lastname" value="Mouse">
```

```
  <br><br>
```

```
  <input type="submit" value="Submit">
```

```
</form>
```

<p>If you click the "Submit" button, the form-data will be sent to a page called "/action_page.php".</p>

```
</body>
```

```
</html>
```

Ejercicio 1

The screenshot shows a web browser window with a form and its developer tools. The form, titled "Submitted Form Data", displays the input: "firstname=Mickey&lastname=Mouse". Below the form, a message states: "Your input was received as: firstname=Mickey&lastname=Mouse". A note indicates that the tutorial does not teach how servers process input. The developer tools are open to the "Network" tab, showing a list of requests. The selected request is "action_page.php?firstname=Mickey&lastname=Mouse". The "Headers" panel shows the request method as "GET", status code as "200", and content type as "text/html". The "Response Headers" panel shows "cache-control: public", "content-encoding: gzip", "content-length: 574", "content-type: text/html", "date: Thu, 08 Mar 2018 07:50:48 GMT", and "server: Microsoft-IIS/7.5".

```
<!DOCTYPE html>
<html>
<body>

<form action="/action_page.php">
  First name:<br>
  <input type="text" name="firstname" value="Mickey">
  <br>
  Last name:<br>
  <input type="text" name="lastname" value="Mouse">
  <br>
  <input type="submit" value="Submit">
</form>

<p>If you click the "Submit" button, the form-data will be sent to a page called
"/action_page.php".</p>
</body>
</html>
```

Submitted Form Data

Your input was received as:

firstname=Mickey&lastname=Mouse

The server has processed your input and returned this answer.

Note: This tutorial will not teach you how servers are processing input. Processing input is explained in our [PHP tutorial](#).

Request Headers:

- Request URL: https://www.w3schools.com/action_page.php?firstname=Mickey&lastname=Mouse
- Request Method: GET
- Status Code: 200
- Remote Address: 192.229.133.221:443
- Referer Policy: no-referrer-when-downgrade

Response Headers:

- cache-control: public
- content-encoding: gzip
- content-length: 574
- content-type: text/html
- date: Thu, 08 Mar 2018 07:50:48 GMT
- server: Microsoft-IIS/7.5

1. Prueba con el formulario de ejemplo
2. Prueba con y sin method="GET"
3. Cambia los datos e identifica el lugar en el que se mandan al servidor
4. Comprueba que el método de llamada es GET

Ejercicio 2

1. Vuelve a probar con el formulario de ejemplo
2. Prueba poniendo method="POST"
3. Cambia los datos e identifica el lugar en el que se mandan al servidor
4. Comprueba que el método de llamada es POST
5. Identifica todos los cambios en las cabeceras.
¿Se corresponde con lo explicado en clase?

Submitted Form Data

Your input was received as:

firstname=Mickey&lastname=Mouse

The server has processed your input and returned this answer.

Note: This tutorial will not teach you how servers are processing input. Processing input is explained in our [PHP tutorial](#).

The screenshot shows the 'Network' tab of a web browser's developer tools. A request to 'action_page.php' is selected. The 'Headers' pane is open, showing the following details:

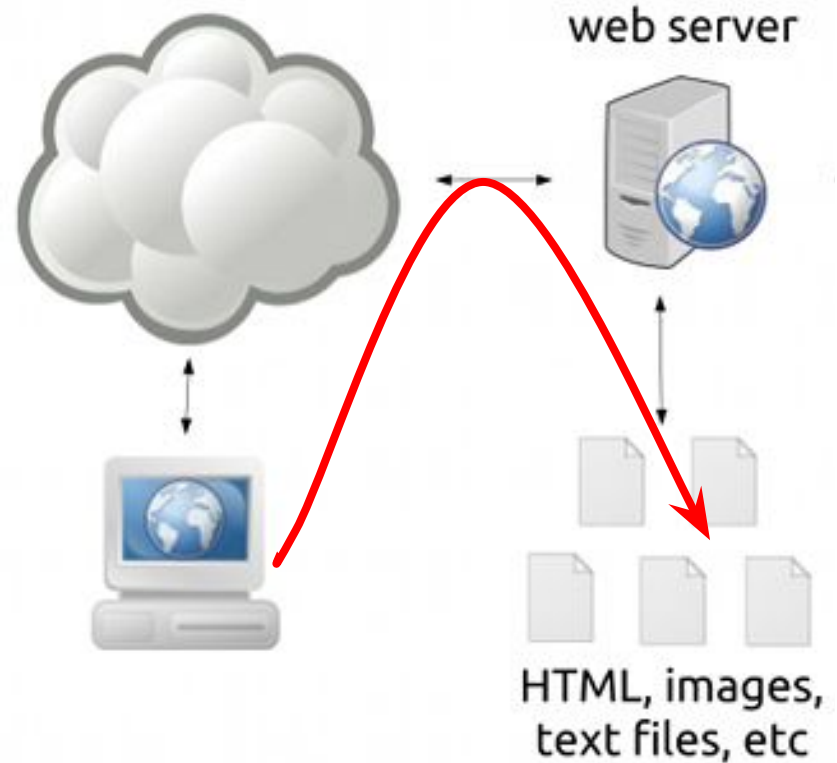
- General:**
 - Request URL: `https://www.w3schools.com/action_page.php`
 - Request Method: **POST**
 - Cache Control: `no-cache`
 - Referer Policy: `no-referrer-when-downgrade`
- Response Headers:**
 - cache-control: `public`
 - content-encoding: `gzip`
 - content-length: `576`
 - content-type: `text/html`
 - date: `Thu, 08 Mar 2018 08:04:06 GMT`
 - server: `Microsoft-IIS/7.5`
 - status: `200`
 - vary: `Accept-Encoding`
 - x-frame-options: `SAMEORIGIN`
 - x-powered-by: `PHP/5.4.2`
 - x-powered-by: `ASP.NET`
- Request Headers:**
 - authority: `www.w3schools.com`
 - method: `POST`
 - path: `/action_page.php`
 - scheme: `https`
 - accept: `text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/png,*/*;q=0.8`
 - accept-encoding: `gzip, deflate, br`
 - accept-language: `es-ES,en;q=0.9,es;q=0.8,fr;q=0.7`
 - cache-control: `max-age=0`
 - content-length: `31`
 - content-type: `application/x-www-form-urlencoded`

The status bar at the bottom indicates: 1 requests | 631.8 transferred | Finish: 162 ms | DOMContentLoaded: 517 ms | Load: 521 ms.

PUT

Características:

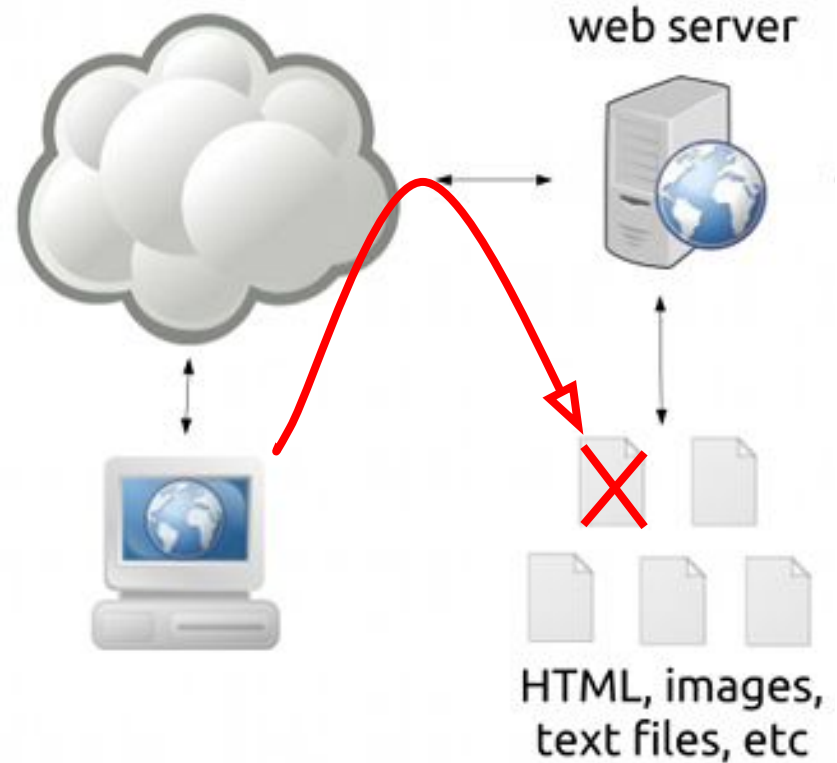
- Lo usamos para actualizar información
- Muy parecido al POST. Pero se supone que ya hay información y queremos actualizar



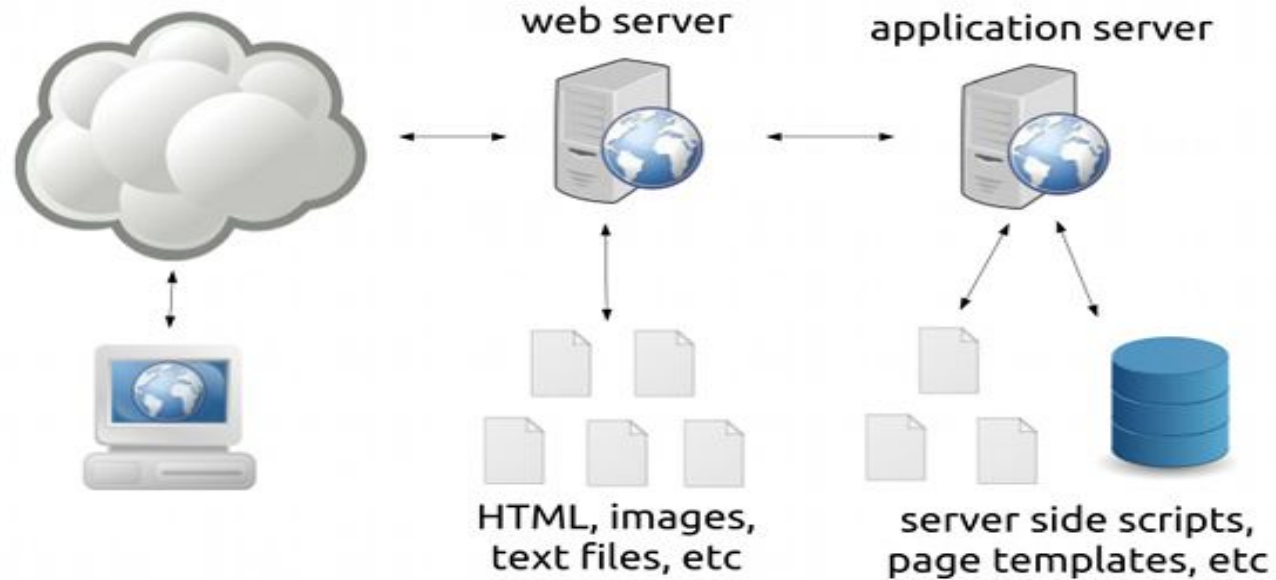
DELETE

Características:

- Lo usamos para borrar información
- No es idempotente
- No se puede cachear



De ficheros a recursos



En un servidor web no solo existen ficheros html o imágenes

También tenemos datos (tu saldo bancario, los datos de tus facturas, tu correo, etc.)

HTML vs XML o JSON

Para las páginas web usamos HTML

Para los datos usamos otros
formatos

1. XML
2. JSON

XML

```
<empinfo>
  <employees>
    <employee>
      <name>James Kirk</name>
      <age>40</age>
    </employee>
    <employee>
      <name>Jean-Luc Picard</name>
      <age>45</age>
    </employee>
    <employee>
      <name>Wesley Crusher</name>
      <age>27</age>
    </employee>
  </employees>
</empinfo>
```

JSON

```
{ "empinfo" :
  {
    "employees" : [
      {
        "name" : "James Kirk",
        "age" : 40,
      },
      {
        "name" : "Jean-Luc Picard",
        "age" : 45,
      },
      {
        "name" : "Wesley Crusher",
        "age" : 27,
      }
    ]
  }
}
```

Usando datos con http

Cada verbo lo usamos para coger, modificar o borrar información.

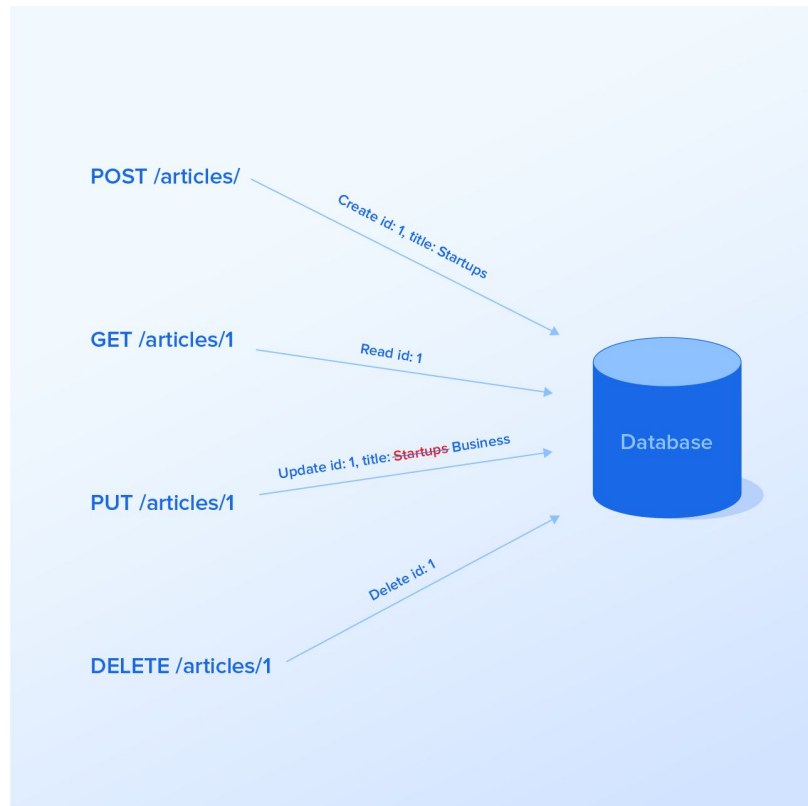
Las URLs representan información y no páginas.

La información viaja en formatos JSON (o XML). No se usa HTML

La funcionalidad se suele agrupar por “servicios”. Por ejemplo, la gestión de clientes se representa por una familia de URL

A todo esto se le llama “Servicios Web”.

Hay dos grandes tipos de servicios Web: SOAP y RESTful



JSON

JSON: JavaScript Object Notation.

- Es un formato para la representación de datos en forma de texto
- Permite almacenar y transmitir información
- Es un formato muy cómodo para trabajar con JavaScript
- Asocia una etiqueta a cada dato
- Permite representar datos de diferentes naturaleza (enteros, objetos, arrays, etc.)
- Mucha más información en https://www.w3schools.com/js/js_json_intro.asp

Ejemplos JSON

```
[{
  "id": 1,
  "first_name": "Jeanette",
  "last_name": "Penddreth",
  "email": "jpenddreth0@census.gov",
  "gender": "Female",
  "ip_address": "26.58.193.2"
}, {
  "id": 2,
  "first_name": "Giavani",
  "last_name": "Frediani",
  "email": "gfrediani1@senate.gov",
  "gender": "Male",
  "ip_address": "229.179.4.212"
}]
```

```
{
  "colors": [
    {
      "color": "black",
      "category": "hue",
      "type": "primary",
      "code": {
        "rgba": [255,255,255,1],
        "hex": "#000"
      }
    },
    {
      "color": "white",
      "category": "value",
      "code": {
        "rgba": [0,0,0,1],
        "hex": "#FFF"
      }
    }
  ]
}
```

JSON con Python



Reading and Writing JSON Data

Problem

You want to read or write data encoded as JSON (JavaScript Object Notation).

Solution

The `json` module provides an easy way to encode and decode data in JSON. The two main functions are `json.dumps()` and `json.loads()`, mirroring the interface used in other serialization libraries, such as `pickle`. Here is how you turn a Python data structure into JSON:

```
import json

data = {
    'name' : 'ACME',
    'shares' : 100,
    'price' : 542.23
}

json_str = json.dumps(data)
```

Here is how you turn a JSON-encoded string back into a Python data structure:

```
data = json.loads(json_str)
```

If you are working with files instead of strings, you can alternatively use `json.dump()` and `json.load()` to encode and decode JSON data. For example:

```
# Writing JSON data
with open('data.json', 'w') as f:
    json.dump(data, f)
```

Material y ejemplos en el libro Python Cookbook.

Si buscas en google encontrarás numerosos tutoriales. Por ejemplo: <http://stackabuse.com/reading-and-writing-json-to-a-file-in-python/>

Mira sobre todo los métodos

- `json.load`,
- `json.dumps`,

```
import json
```

```
data = {
    'name' : 'ACME',
    'shares' : 100,
    'price' : 542.23
}
```

```
json_str = json.dumps(data)
```

Ejercicio 3

1. Haz un programa Python que cree un JSON con datos de tu cuenta y que lo salve en un fichero
 - a. Por ejemplo, graba en un JSON tu nombre, tus apellidos, el nombre de tu máquina, la IP de tu máquina, el directorio home de tu cuenta, etc. Cualquier cosa que te parezca relevante.
 - b. Utiliza un formato JSON apropiado. Cada campo con su nombre y con su valor.
 - c. Utiliza los ejemplos que hay en las transparencias
 - d. Graba toda esa información en un fichero (por ejemplo: "misdatos.json")
2. Manda ese fichero a un compañero y pídele que lo lea con un programa Python. Ese programa debe imprimir por pantalla tu nombre y tu IP
 - a. Puedes mandar ese fichero por correo electrónico o se lo copias a un USB. Esa parte no es importante
 - b. Lo importante es que un programa hecho por tu compañero sea capaz de leer una información que te has inventado tu.

Ejercicio 4

1. Haz un servidor web en Python que retorne el JSON de la práctica anteriores
 - a. Cuando pidas <http://127.0.0.1:8000/index.html> te devuelve una página web
 - b. Pero cuando pidas <http://127.0.0.1:8000>datos.json te devuelve el fichero json
 - c. Recuerda que tiene que cambiar el content type ...
2. Haz un programa que se conecte a tu servidor y que imprima por pantalla tu nombre y tu IP
 - a. Tienes que hacer un cliente. Mira alguna librería en Python que te pueda ser útil
3. Utiliza tu programa cliente para imprimir los datos que te ofrezca el servidor de un compañero
 - a. Lo importante es que tu programa se baje los datos de tu compañero y que sea capaz de leer su información.